TASC LMS Frontend — Pending Tasks (Final Specification)

> **For Claude Code:** Tasks are organized by priority with exact file paths, component names, API service methods, hook names, query keys, and TypeScript interfaces from the actual codebase. Each task tells you exactly what to import, what to call, and what response shape to expect. Work top-down.

Reference: Frontend Patterns

Data Flow

```
Page Component
  → useQuery/useMutation hook (src/hooks/)
    → service function (src/services/*.services.ts)
      → apiClient (src/utils/config.ts) with auth interceptor
        → Backend API
```

Adding a New API Integration

1. **Service method** exists in `src/services/*.services.ts`? If not, add it.
2. **Query key** exists in `src/hooks/queryKeys.ts`? If not, add it.
3. **Hook** exists in `src/hooks/use*.ts`? If not, add it.
4. **Page** calls the hook, destructures `{ data, isLoading, error }`, replaces hardcoded arrays.

Response Pattern

```
// All list endpoints return PaginatedResponse<T>
const { data } = useQuery({ queryKey: [...], queryFn: () => api.getAll().then(r => r.data) });
const items = data?.results ?? [];
const total = data?.count ?? 0;
```

Key Service Objects (from `src/services/main.api.ts`)

```
api.catalogue.course     // courseApi
api.catalogue.session    // sessionApi
api.catalogue.category   // categoryApi
api.learning.enrollment  // enrollmentApi
api.learning.certificate // certificateApi
api.learning.discussion  // discussionApi
api.payments.invoice     // invoiceApi
api.payments.transaction // transactionApi
api.payments.subscription // subscriptionApi
api.payments.userSubscription // userSubscriptionApi
api.payments.paymentMethod // paymentMethodApi
api.notifications        // notificationApi
api.reports              // reportsApi
api.users                // usersApi
api.upload               // uploadApi
api.livestream.session   // livestreamApi
api.messaging            // messagingApi
```

Quick Summary — Open Tasks

Pri	#	Task	File(s)	What To Do
HIGH	F1	Saved Courses page + toggle	`SavedCoursesPage.tsx`, `CatalogCourseCard.tsx`	New `savedCourseApi` + hooks, replace state
HIGH	F2	Finance pages — replace mock data	8 finance page files	Replace `const data = [...]` with existing hooks
HIGH	F3	Superadmin pages — wire to stats APIs	7 superadmin page files	New service methods + hooks for stats endpoints
HIGH	F4	Mobile money checkout	`CheckoutPaymentPage.tsx`	Replace `setTimeout` with real Flutterwave charge
HIGH	F5	Promo code validation	`CheckoutPaymentPage.tsx`, `SubscriptionManagementPage.tsx`	New `promoCodeApi`, wire to input
MED	F6	Learner Progress page overhaul	`ProgressPage.tsx`	Replace hardcoded data with `enrollmentApi` + `sessionProgressApi`
MED	F7	Instructor Grading page wiring	`GradingPage.tsx`	Replace `sampleSubmissions` with `submissionApi`
MED	F8	Manager Settings persistence	`ManagerSettingsPage.tsx`	Wire to new `GET/PUT /api/v1/manager/settings/`
MED	F9	Manager Billing real data	`ManagerBillingPage.tsx`	Wire plan/usage to new endpoints
MED	F10	Review interactions	`CourseReviews.tsx`	Wire Helpful/Report buttons to new review actions
MED	F11	CourseViewSet ordering (TopCourses)	`ManagerDashboard.tsx`	Use `ordering` param once backend Task 30 done
MED	F12	Catalog sorting	`CourseCataloguePage.tsx`	Wire Sort dropdown to `publicCourseApi` `ordering` param
LOW	F13	Business page APIs	`PricingSection.tsx`, `FaqSection.tsx`	Wire to `businessPricingApi` / `faqApi`
LOW	F14	Invoice PDF/email	`InvoiceReceiptPage.tsx`	Wire Download/Email buttons
LOW	F15	Payment history exports	`PaymentHistoryPage.tsx`	Wire Export CSV / Statement buttons
LOW	F16	Subscription actions	`SubscriptionManagementPage.tsx`	Wire Cancel/Add Payment mutations
LOW	F17	Security page	`SecurityPage.tsx`	Wire to new security stats endpoint
LOW	F18	Dashboard mock cleanup	various	Remove remaining hardcoded stats

HIGH PRIORITY

Task F1: Saved Courses — Full Stack Wiring

Problem: `SavedCoursesPage.tsx` at route `/learner/saved` shows 6 hardcoded courses. `CatalogCourseCard.tsx` heart icon uses React `useState` — lost on refresh.

Backend dependency: Task 29 (new `SavedCourse` model + endpoints)

Step 1 — Add service methods (`src/services/learning.services.ts`, append):

```
export interface SavedCourseResponse {
  id: number;
  course: CourseList;
  created_at: string;
}

export const savedCourseApi = {
  getAll: (params?: { page?: number; page_size?: number }) =>
    apiClient.get<PaginatedResponse<SavedCourseResponse>>('/api/v1/learning/saved-courses/', {
params }),

  save: (courseId: number) =>
    apiClient.post('/api/v1/learning/saved-courses/', { course: courseId }),

  unsave: (id: number) =>
    apiClient.delete(`/api/v1/learning/saved-courses/${id}/`),

  toggle: (courseId: number) =>
    apiClient.post<{ saved: boolean }>('/api/v1/learning/saved-courses/toggle/', { course:
courseId }),
};
```

Step 2 — Add to consolidated export (`src/services/main.api.ts`):

```
import { savedCourseApi } from './learning.services';
// In the api object, under learning:
learning: {
  // ...existing...
  savedCourse: savedCourseApi,
},
```

Step 3 — Add query key (`src/hooks/queryKeys.ts`):

```
savedCourses: {
  all: ['saved-courses'] as const,
},
```

Step 4 — Add hooks (`src/hooks/useLearning.ts`, append):

```
export const useSavedCourses = () =>
  useQuery({
    queryKey: queryKeys.savedCourses.all,
    queryFn: () => savedCourseApi.getAll().then(r => r.data),
  });

export const useToggleSavedCourse = () => {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (courseId: number) => savedCourseApi.toggle(courseId).then(r => r.data),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: queryKeys.savedCourses.all });
    },
  });
};
```

Step 5 — Wire `SavedCoursesPage.tsx`:

Replace the hardcoded `const savedCourses = [...]` with:

```
import { useSavedCourses } from '../../hooks/useLearning';

const SavedCoursesPage = () => {
  const { data, isLoading } = useSavedCourses();
  const courses = data?.results ?? [];

  // Replace the hardcoded grid with courses.map(sc => ...)
  // sc.course contains the full CourseList object
};
```

Step 6 — Wire `CatalogCourseCard.tsx` or `LearnerCourseCatalogPage.tsx`:

Replace `const [favorites, setFavorites] = useState<Set<number>>(new Set())` with:

```
import { useSavedCourses, useToggleSavedCourse } from '../../hooks/useLearning';

const { data: savedData } = useSavedCourses();
const savedIds = new Set(savedData?.results?.map(sc => sc.course.id) ?? []);
const toggleMutation = useToggleSavedCourse();

// In the heart icon onClick:
const handleFavorite = (courseId: number) => {
  toggleMutation.mutate(courseId);
};

// For isFavorite:
const isFavorite = savedIds.has(course.id);
```

Task F2: Finance Pages — Replace Mock Data with Existing APIs

Problem: 8 finance pages have hardcoded `const data = [...]` arrays despite the API services and hooks already existing.

Section F from back.md confirms these pages have inline mock data:

```
src/pages/finance/FinancePaymentsPage.tsx:30:const payments: Payment[] = [
src/pages/finance/FinanceInvoicesPage.tsx:30:const invoices: Invoice[] = [
src/pages/finance/FinanceSubscriptionsPage.tsx:33:const subscriptions: Subscription[] = [
src/pages/finance/FinanceRevenueReportsPage.tsx:17:const revenueKpis = [
src/pages/finance/FinanceSubscriptionHistoryPage.tsx:29:const events: HistoryEvent[] = [
src/pages/finance/FinanceChurnPage.tsx:13:const churnKpis = [
src/pages/finance/FinanceStatementsPage.tsx:40:const balanceItems = [
src/pages/finance/FinanceCustomReportsPage.tsx:37:const reports: CustomReport[] = [
```

For each page, the pattern is identical:

F2a: `FinancePaymentsPage.tsx`

Delete: `const payments: Payment[] = [...]` (line 30)

Replace with:

```
import { useTransactions } from '../../hooks/usePayments';

const FinancePaymentsPage = () => {
  const [params, setParams] = useState<TransactionParams>({});
  const { data, isLoading } = useTransactions(params);
  const payments = data?.results ?? [];
  // ... rest of component uses payments array
};
```

Hooks exist: `useTransactions` (`src/hooks/usePayments.ts:105`) **Service exists:** `transactionApi.getAll()` (`src/services/payments.services.ts:82`) **Query key exists:** `queryKeys.transactions.all(params)` → `['transactions', params]`

F2b: `FinanceInvoicesPage.tsx`

Delete: `const invoices: Invoice[] = [...]` (line 30)

Replace with:

```
import { useInvoices } from '../../hooks/usePayments';

const { data, isLoading } = useInvoices(params);
const invoices = data?.results ?? [];
```

Hooks exist: `useInvoices` (`src/hooks/usePayments.ts:26`) **Service exists:** `invoiceApi.getAll()` (`src/services/payments.services.ts:33`)

F2c: `FinanceSubscriptionsPage.tsx`

Delete: `const subscriptions: Subscription[] = [...]` (line 33)

Replace with:

```
import { useSubscriptions, useUserSubscriptions } from '../../hooks/usePayments';

const { data: plansData } = useSubscriptions();
const plans = plansData?.results ?? [];

const { data: userSubsData } = useUserSubscriptions();
const userSubs = userSubsData?.results ?? [];
```

Hooks exist: `useSubscriptions` (`usePayments.ts:217`), `useUserSubscriptions` (`usePayments.ts:241`)

F2d: `FinanceRevenueReportsPage.tsx`

Delete: `const revenueKpis = [...]` (line 17), `const monthlyBreakdown = [...]` (line 25), etc.

Replace with:

```
import { useRevenueTrends } from '../../services/learning.services';
// useRevenueTrends is exported from learning.services.ts line 463

const { data: revenueTrends, isLoading } = useRevenueTrends(12);
```

Note: `useRevenueTrends` is defined at `learning.services.ts:463` as a hook (unusual — it's in the services file). It calls `GET /api/v1/payments/analytics/revenue/`.

F2e–F2h: Remaining Finance Pages

Same pattern. For each:

1. Find the `const hardcodedData = [...]` declaration
2. Delete it
3. Import the appropriate hook
4. Replace with `const { data, isLoading } = useXxx()`
5. Use `data?.results ?? []` or direct `data` object

Pages with NO existing backend endpoint (needs backend work first):

- `FinanceChurnPage.tsx` — No churn analytics endpoint exists → Keep mock, add TODO
- `FinanceStatementsPage.tsx` — No financial statements endpoint → Keep mock, add TODO
- `FinanceCustomReportsPage.tsx` — Uses `reportsApi` already for types, but custom report list is mocked → Wire to `reportsApi.getAll()`

Task F3: Superadmin Pages — Wire to Stats APIs

Problem: Multiple superadmin pages are entirely mock data. Backend Tasks 32–37 create the stats endpoints.

Pages and their backend dependencies:

Page File	Backend Task	Endpoint to Call	Service to Create
`AllCoursesPage.tsx`	32	`GET /api/v1/catalogue/courses/stats/`	`courseApi.getStats()`
`AssessmentsPage.tsx`	33	`GET /api/v1/superadmin/assessments/stats/`	new `assessmentApi.getStats()`
`CertificationsPage.tsx`	34	`GET /api/v1/learning/certificates/stats/`	`certificateApi.getStats()`
`InstructorsPage.tsx`	35	`GET /api/v1/superadmin/users/instructor-stats/`	`usersApi.getInstructorStats()`
`InvoicesPage.tsx`	36	`GET /api/v1/payments/invoices/stats/`	`invoiceApi.getStats()`
`RevenuePage.tsx`	37	`GET /api/v1/payments/revenue/stats/`	new `revenueApi.getStats()`
`RolesPermissionsPage.tsx`	40	`GET /api/v1/superadmin/users/stats/` (enhanced)	`userStatsApi.getStats()` (already exists)

Example wiring for `AllCoursesPage.tsx`:

Step 1 — Add service method (`src/services/catalogue.services.ts`):

```
// Add to courseApi object:
getStats: () =>
  apiClient.get<{ total: number; published: number; draft: number; archived: number }>(
    '/api/v1/catalogue/courses/stats/'
  ),
```

Step 2 — Add hook (`src/hooks/useCatalogue.ts`):

```
export const useCourseStats = () =>
  useQuery({
    queryKey: ['courses', 'stats'] as const,
    queryFn: () => courseApi.getStats().then(r => r.data),
  });
```

Step 3 — Wire page (`src/pages/superadmin/AllCoursesPage.tsx`):

Find the hardcoded KPI object and replace:

```
import { useCourseStats, useCourses } from '../../hooks/useCatalogue';

const { data: stats, isLoading: statsLoading } = useCourseStats();
const { data: coursesData, isLoading: coursesLoading } = useCourses(tableParams);

// KPI cards:
const kpis = [
  { label: 'Total Courses', value: stats?.total ?? 0 },
  { label: 'Published', value: stats?.published ?? 0 },
  { label: 'Draft', value: stats?.draft ?? 0 },
  { label: 'Archived', value: stats?.archived ?? 0 },
];

// Table:
const courses = coursesData?.results ?? [];
```

Repeat this pattern for each superadmin page. The table data usually comes from existing list endpoints (`courseApi.getAll()`, `usersApi.getAll()`, `invoiceApi.getAll()`), and only the KPI stats need new endpoints.

For `InstructorsPage.tsx`:

`usersApi.getInstructorStats()` already exists (`users.services.ts:85`). It returns `Promise<InstructorStats>` with `{ total, active, avg_rating, total_courses }`. Just call it:

```
import { usersApi, InstructorStats } from '../../services/users.services';

const { data: stats } = useQuery({
  queryKey: ['instructors', 'stats'],
  queryFn: () => usersApi.getInstructorStats(),
});
```

For the instructors table, use:

```
const { data: instructorsData } = useQuery({
  queryKey: ['users', { role: 'instructor' }],
  queryFn: () => usersApi.getInstructors({ role: 'instructor' }).then(r => r.data),
});
```

Task F4: Mobile Money Checkout — Replace setTimeout

File: `src/pages/learner/CheckoutPaymentPage.tsx`

Problem: Mobile money payment uses `setTimeout(() => { setPaymentStatus('success') }, 3000)` to simulate payment.

Backend dependency: Task 60 (Flutterwave mobile money charge endpoint)

Step 1 — Add service method (`src/services/payments.services.ts`):

```
export interface MobileMoneyChargeRequest {
  enrollment: number;
  phone_number: string;
  provider: 'mpesa' | 'mtn' | 'airtel';
  promo_code?: string;
}

export interface MobileMoneyChargeResponse {
  status: 'pending' | 'error';
  message: string;
  flw_ref?: string;
}

export const mobileMoneyApi = {
  charge: (data: MobileMoneyChargeRequest) =>
    apiClient.post<MobileMoneyChargeResponse>(
      '/api/v1/payments/flutterwave/charge-mobile-money/',
      data
    ),
};
```

Step 2 — Wire in `CheckoutPaymentPage.tsx`:

Find the `setTimeout` block in the mobile money handler and replace:

```
import { mobileMoneyApi } from '../../services/payments.services';

const handleMobileMoneyPayment = async () => {
  setPaymentStatus('processing');
  try {
    const response = await mobileMoneyApi.charge({
      enrollment: enrollmentId,
      phone_number: phoneNumber,
      provider: selectedProvider as 'mpesa' | 'mtn' | 'airtel',
      promo_code: promoCode || undefined,
    });
    if (response.data.status === 'pending') {
      setPaymentStatus('pending');
      // Show "Check your phone" message
      // Poll for completion or wait for webhook redirect
    }
  } catch (error) {
    setPaymentStatus('failed');
    // Show error toast
  }
};
```

Task F5: Promo Code Validation

Files:

- `src/pages/learner/CheckoutPaymentPage.tsx` — hardcoded "SAVE20" acceptance
- `src/pages/learner/SubscriptionManagementPage.tsx` — same

Backend dependency: Task 61 (PromoCode model + verify endpoint)

Step 1 — Add service (`src/services/public.services.ts`):

```
export interface PromoCodeVerifyResponse {
  valid: boolean;
  code?: string;
  discount_percent?: number;
  discount_amount?: string;
  error?: string;
}

export const promoCodeApi = {
  verify: (code: string, courseId?: number) =>
    apiClient.get<PromoCodeVerifyResponse>('/api/v1/public/promo-codes/verify/', {
      params: { code, course: courseId },
    }),
};
```

Step 2 — Wire in `CheckoutPaymentPage.tsx`:

Find the promo code input handler (where it checks `code === 'SAVE20'`) and replace:

```
const handleApplyPromo = async () => {
  try {
    const { data } = await promoCodeApi.verify(promoCode, courseId);
    if (data.valid) {
      setDiscount(data.discount_percent || 0);
      setDiscountAmount(data.discount_amount || '0');
      showToast('Promo code applied!', 'success');
    } else {
      showToast(data.error || 'Invalid promo code', 'error');
    }
  } catch {
    showToast('Invalid promo code', 'error');
  }
};
```

MEDIUM PRIORITY

Task F6: Learner Progress Page — Full Overhaul

File: `src/pages/learner/ProgressPage.tsx`

Problem: 0% backend integration. All stats, course progress lists, milestones are hardcoded.

APIs that already exist for this:

- `enrollmentApi.getAll()` → `useEnrollments()` — user's enrollments with `progress_percentage`, `status`
- `sessionProgressApi.getAll({ enrollment })` → `useSessionProgressList()` — per-session completion
- `analyticsApi.getLearningStats()` → from `learning.services.ts:303`

Do this:

```
import { useEnrollments } from '../../hooks/useLearning';
import { useLearningStats } from '../../services/learning.services';
```

```
const ProgressPage = () => {
  const { data: enrollmentsData, isLoading } = useEnrollments();
  const enrollments = enrollmentsData?.results ?? [];
  const { data: stats } = useLearningStats(); // returns LearningStats

  // Compute from real data:
  const completedCourses = enrollments.filter(e => e.status === 'completed').length;
  const activeCourses = enrollments.filter(e => e.status === 'active').length;
  const avgProgress = enrollments.reduce((sum, e) => sum + e.progress_percentage, 0)
    / (enrollments.length || 1);

  // For per-course progress cards, map enrollments:
  const courseProgress = enrollments.map(e => ({
    courseTitle: e.course_title,
    thumbnail: e.course_thumbnail,
    progress: e.progress_percentage,
    status: e.status,
    lastAccessed: e.last_accessed_at,
  }));
};
```

Enrollment interface fields available (from `types.ts`):

- `progress_percentage: number`
- `status: EnrollmentStatus` (active, completed, dropped, expired)
- `course_title: string`
- `course_thumbnail: string`
- `last_accessed_at: string`
- `time_remaining_days: number`
- `certificate_issued: boolean`

Task F7: Instructor Grading Page — Wire to Submissions API

File: `src/pages/instructor/GradingPage.tsx`

Problem: Uses `sampleSubmissions: SubmissionData[] = [...]` (line 25) and `sampleFiles: FileData[] = [...]` (line 73).

APIs that already exist:

- `submissionApi.getAll()` → `useSubmissions()` hook (`src/hooks/useSubmissions.ts:16`)
- `submissionApi.grade(id, data)` → `useGradeSubmission()` hook (`src/hooks/useSubmissions.ts:66`)

Do this:

```
import { useSubmissions, useGradeSubmission } from '../../hooks/useSubmissions';

const GradingPage = () => {
  const { data: subsData, isLoading } = useSubmissions({
    status: 'submitted',  // Only show ungraded submissions
  });
  const submissions = subsData?.results ?? [];
  const gradeMutation = useGradeSubmission();

  const handleGrade = (submissionId: number, grade: number, feedback: string) => {
    gradeMutation.mutate(
      { id: submissionId, data: { grade, feedback } },
      {
        onSuccess: () => showToast('Graded successfully', 'success'),
      }
```

```
    );
  };
};
```

Submission interface fields (from `types.ts`):

- `id`, `assignment_title`, `session_title`, `user_name`, `user_email`
- `status: SubmissionStatus` (draft, submitted, graded, pending_review, rejected)
- `submitted_file_url`, `submitted_file_name`, `submitted_text`
- `grade`, `feedback`, `graded_at`, `graded_by_name`

Delete `sampleSubmissions` and `sampleFiles` arrays entirely.

Task F8: Manager Settings — Wire to Backend

File: `src/pages/manager/ManagerSettingsPage.tsx`

Problem: Entire page is a static mock. No backend persistence.

Backend dependency: Task 43 (`GET/PUT /api/v1/manager/settings/`)

Step 1 — Add service (`src/services/organization.services.ts` or new file):

```
export interface ManagerSettings {
  org_name: string;
  description: string;
  logo: string | null;
  website: string | null;
  contact_email: string | null;
  contact_phone: string | null;
  address: string;
  city: string;
  country: string;
}

export const managerSettingsApi = {
  get: () => apiClient.get<ManagerSettings>('/api/v1/manager/settings/'),
  update: (data: Partial<ManagerSettings>) =>
    apiClient.put<ManagerSettings>('/api/v1/manager/settings/', data),
};
```

Step 2 — Wire page:

```
const { data: settings, isLoading } = useQuery({
  queryKey: ['manager', 'settings'],
  queryFn: () => managerSettingsApi.get().then(r => r.data),
});

const updateMutation = useMutation({
  mutationFn: (data: Partial<ManagerSettings>) =>
    managerSettingsApi.update(data).then(r => r.data),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['manager', 'settings'] });
    showToast('Settings saved', 'success');
  },
});
```

Task F9: Manager Billing — Wire Real Data

File: `src/pages/manager/ManagerBillingPage.tsx`

Problem: Plan info, usage stats, and payment method are mocked. Invoice table may be wired.

Backend dependency: Task 44 (`GET /api/v1/manager/billing/plan/` and `/usage/`)

Step 1 — Add services:

```
export const managerBillingApi = {
  getPlan: () => apiClient.get('/api/v1/manager/billing/plan/'),
  getUsage: () => apiClient.get('/api/v1/manager/billing/usage/'),
};
```

Step 2 — Wire page sections:

```
const { data: plan } = useQuery({
  queryKey: ['manager', 'billing', 'plan'],
  queryFn: () => managerBillingApi.getPlan().then(r => r.data),
});

const { data: usage } = useQuery({
  queryKey: ['manager', 'billing', 'usage'],
  queryFn: () => managerBillingApi.getUsage().then(r => r.data),
});
```

Replace hardcoded "Enterprise $499/mo" with `plan?.plan_name` and `plan?.price`. Replace hardcoded "347 users" with `usage?.active_users`.

Task F10: Course Review Interactions

File: `src/components/course/CourseReviews.tsx` (or similar in `components/learner/`)

Problem: "Helpful" and "Report" buttons are non-functional. Star filter dropdown is ignored.

Backend dependency: Task 62 (review `helpful` and `report` actions, `rating` filter)

Step 1 — Add service methods (`src/services/catalogue.services.ts`):

```
// Add to courseReviewApi:
helpful: (reviewId: number) =>
  apiClient.post(`/api/v1/catalogue/reviews/${reviewId}/helpful/`),

report: (reviewId: number) =>
  apiClient.post(`/api/v1/catalogue/reviews/${reviewId}/report/`),
```

Step 2 — Wire buttons:

```
const handleHelpful = async (reviewId: number) => {
  await courseReviewApi.helpful(reviewId);
  showToast('Marked as helpful', 'success');
};

const handleReport = async (reviewId: number) => {
```

```
  await courseReviewApi.report(reviewId);
  showToast('Review reported', 'info');
};
```

Step 3 — Wire rating filter:

The `courseReviewApi.getAll()` already accepts `{ rating?: number }` param (line 370). Just pass it:

```
const [ratingFilter, setRatingFilter] = useState<number | undefined>();
const { data: reviewsData } = useQuery({
  queryKey: ['course-reviews', courseId, ratingFilter],
  queryFn: () => courseReviewApi.getAll({ course: courseId, rating: ratingFilter }).then(r =>
r.data),
});
```

Task F11: Manager Dashboard — TopCourses Widget

File: `src/pages/manager/ManagerDashboard.tsx` (or a `TopCourses` component)

Problem: Fetches first 4 courses in default order instead of most-enrolled.

Backend dependency: Task 30 (add `OrderingFilter` to `CourseViewSet`)

Once backend is ready, change the query:

```
const { data: topCoursesData } = useCourses({
  ordering: '-enrollment_count',
  page_size: 4,
});
```

Note: The `CourseListParams` interface already has no `ordering` field. Add it:

```
// In catalogue.services.ts, CourseListParams:
export interface CourseListParams {
  // ...existing fields...
  ordering?: string;
}
```

Task F12: Public Catalog — Sort Dropdown

File: `src/pages/public/CourseCataloguePage.tsx`

Problem: Sort dropdown exists but `publicCourseApi` doesn't pass `ordering` param.

Backend status: `PublicCourseViewSet` already has `OrderingFilter` configured.

Fix: The `PublicCourseParams` interface (`public.services.ts:52`) doesn't include `ordering`. Add it:

```
export interface PublicCourseParams {
  // ...existing...
  ordering?: string;  // e.g., '-published_at', 'title', '-enrollment_count'
}
```

Then in the page, wire the dropdown:

```
const [sortBy, setSortBy] = useState('-published_at');
const { data } = usePublicCourses({ ...filters, ordering: sortBy });
```

LOW PRIORITY

Task F13: Business Page Components

Files:

- `src/components/business/PricingSection.tsx` — `businessPricingApi.getPlans()` exists but may not be wired
- `src/components/business/FaqSection.tsx` — `faqApi.getAll()` exists in `public.services.ts:149`

Services exist (`public.services.ts`):

```
businessPricingApi.getPlans()  // line 133
faqApi.getAll()                // line 149
```

Wire PricingSection:

```
const { data: plans } = useQuery({
  queryKey: ['business-plans'],
  queryFn: () => businessPricingApi.getPlans().then(r => r.data),
});
```

Wire FaqSection:

```
const { data: faqs } = useQuery({
  queryKey: ['faqs'],
  queryFn: () => faqApi.getAll().then(r => r.data),
});
```

Task F14: Invoice PDF Download & Email

File: `src/pages/learner/InvoiceReceiptPage.tsx`

Problem: "Download PDF" and "Email Invoice" buttons show mock toasts.

Backend dependency: Task 63 (invoice download-pdf and email-receipt endpoints)

Wire:

```
const handleDownloadPDF = async () => {
  try {
    const response = await invoiceApi.getById(invoiceId); // or a dedicated download endpoint
    // Use window.print() with the A4 CSS template already in CertificatePrint.css
    window.print();
  } catch {
    showToast('Failed to download', 'error');
  }
```

```
};

const handleEmailReceipt = async () => {
  try {
    await apiClient.post(`/api/v1/payments/invoices/${invoiceId}/email-receipt/`);
    showToast('Receipt emailed successfully', 'success');
  } catch {
    showToast('Failed to send email', 'error');
  }
};
```

Task F15: Payment History Exports

File: `src/pages/learner/PaymentHistoryPage.tsx`

Problem: "Export CSV" and "Download Statement" buttons are placeholders.

Backend dependency: Task 63 (transaction export-csv endpoint)

Wire:

```
const handleExportCSV = async () => {
  try {
    const response = await apiClient.get('/api/v1/payments/transactions/export-csv/', {
      responseType: 'blob',
    });
    const url = window.URL.createObjectURL(new Blob([response.data]));
    const link = document.createElement('a');
    link.href = url;
    link.setAttribute('download', 'transactions.csv');
    document.body.appendChild(link);
    link.click();
    link.remove();
  } catch {
    showToast('Export failed', 'error');
  }
};
```

Task F16: Subscription Management Actions

File: `src/pages/learner/SubscriptionManagementPage.tsx`

Problem: Cancel subscription, add payment method, set default — all show placeholder toasts.

Hooks exist:

- `useCancelUserSubscription()` (`usePayments.ts:307`)
- `useCreatePaymentMethod()` (`usePayments.ts:139`)
- `useSetDefaultPaymentMethod()` (`usePayments.ts:204`)

Wire:

```
const cancelMutation = useCancelUserSubscription();
const createPaymentMutation = useCreatePaymentMethod();
const setDefaultMutation = useSetDefaultPaymentMethod();

const handleCancelSubscription = () => {
```

```
    cancelMutation.mutate(subscriptionId, {
      onSuccess: () => showToast('Subscription cancelled', 'info'),
    });
  };
```

Task F17: Security Page

File: `src/pages/superadmin/SecurityPage.tsx`

Backend dependency: Task 22 (security stats endpoint)

Once backend ready, add service and wire:

```
export const securityApi = {
  getStats: () => apiClient.get('/api/v1/superadmin/security/stats/'),
};

// In the page:
const { data: stats } = useQuery({
  queryKey: ['security', 'stats'],
  queryFn: () => securityApi.getStats().then(r => r.data),
});
```

Replace hardcoded "42 active sessions" with `stats?.active_sessions`, etc.

Task F18: Dashboard Mock Cleanup

Miscellaneous remaining hardcoded values across dashboards:

Component	File	Hardcoded Value	Replace With
Learner Dashboard "Learning Hours"	`LearnerDashboard.tsx`	Static number	Sum `time_spent_seconds` from `sessionProgressApi`
Instructor WelcomeBanner stats	`components/instructor/WelcomeBanner.tsx`	Hardcoded counts	`useLearningStats()` data
Manager KPIGrid	`ManagerDashboard.tsx`	Some KPIs mocked	`useLearningStats()` + `useCourseStats()`
Finance Sidebar revenue	`components/finance/Sidebar.tsx`	`$8,425`	`useRevenueTrends()` current month
Superadmin RevenueChart	`components/superadmin/RevenueChart.tsx`	Mock chart data	`useRevenueTrends()` once backend Task 37 done
Superadmin SystemHealth	`components/superadmin/SystemHealth.tsx`	Mock health data	`securityApi.getHealth()` once backend Task 38 done

For each, the fix is the same pattern: import the hook, call it, replace the hardcoded const.

Gateway Pages — Keep as Mock (No Backend Yet)

These finance gateway pages have no backend support and are acceptable as static displays for now:

- `GatewayMpesaPage.tsx` — hardcoded M-Pesa transactions and health

- `GatewayMtnPage.tsx` — hardcoded MTN transactions and health
- `GatewayAirtelPage.tsx` — hardcoded Airtel transactions and health
- `GatewayPesapalPage.tsx` — hardcoded Pesapal transactions and health

Action: Add `{/* TODO: Wire to gateway health API when available */}` comment at top of each file.

Pages That Need NO Frontend Work (Backend Only)

These pages are blocked purely on backend. Once the backend task is completed, the frontend wiring is trivial (1-2 line hook call):

Page	Route	Blocked By Backend Task
`SuperadminNotificationsPage`	`/superadmin/notifications`	✕ Not blocked — existing `notificationApi` works
`ManagerActivityPage`	`/manager/activity`	Task 45 (audit log summary — optional)
`ManagerRolesPage`	`/manager/roles`	Task 40 (user counts by role)
`PartnershipsPage`	`/superadmin/partnerships`	Task 41 (Phase 2)
`DataMigrationPage`	`/superadmin/data-migration`	Task 42 (Phase 2)
`GatewaySettingsPage`	`/superadmin/gateway-settings`	Task 42 (Phase 2)
`SystemSettingsPage`	`/superadmin/settings`	Task 38 (system settings CRUD)
`IntegrationsPage`	`/superadmin/integrations`	Task 41 (Phase 2)

Completed Items Archive

▶ Click to expand — all completed items

Date	Item
27 Mar	ManagerMessagesPage & LearnerMessagesPage wired to `messagingApi`
27 Mar	ManagerBulkEnrollPage wired to `enrollmentApi.bulkEnroll()`
27 Mar	CoursePlayerPage Resources tab wired to `sessionAttachmentApi`
27 Mar	LearnerCertificatesPage mock data removed — uses auto-created certificates
26 Mar	Manager/Instructor Analytics charts wired to analytics hooks
26 Mar	Finance RevenueChart wired to `useRevenueTrends`
26 Mar	Bulk Import CSV format mismatch fixed
26 Mar	CoursePlayerPage Q&A Pin/Lock wired
26 Mar	LearnerAssignmentsPage validation errors wired
25 Mar	InstructorMessagesPage wired to messaging API
18 Mar	CoursePlayerPage Q&A, WorkshopsPage, LearnersPage, CoursePreviewPage wired
18 Mar	LearnerCourseDetailPage, ManagerRecordingsPage, ManagerScheduleNewPage wired
18 Mar	SuperadminPaymentsPage, FinanceAnalyticsPage, CourseReviews wired
17 Mar	Course publish flow, Learner assignments, Bulk import drag-drop
16 Mar	Drag-and-drop reordering, Quiz player, Video resume, QuizzesPage
16 Mar	NotificationsPage, Notes tab persistence

Date	Item
15 Mar	Certificate/invoice download
14 Mar	Privacy policy + certificate validation pages
13 Mar	Initial manager pages wiring
—	Landing Categories wired to `publicCategoryApi`
—	Landing Featured Courses wired to `publicCourseApi`
—	Landing Pricing wired to `publicSubscriptionPlansApi`
—	Landing StatsBanner wired to `publicStatsApi`
—	TrustedBy wired to `publicClientsApi`
—	All auth pages (Login, Register, MFA, Email Verify, Password Reset) fully wired
—	CoursePlayerPage (Video, Documents, Quizzes, Notes, Q&A, Progress) fully wired
—	QuestionBankPage fully wired
—	AssignmentsHubPage fully wired
—	GradebookPage wired to `submissionApi` + `gradeStatisticsApi`
—	ContentUploadPage StorageInfoCard wired to `quotaApi`
—	ManagerCategoriesPage CRUD functional
—	CourseApprovalPage + Detail functional
—	ManagerReportsPage wired to `reportsApi`
—	FinanceExportPage wired to `reportsApi`
—	FinanceProfilePage wired
—	SuperadminProfilePage wired
—	InviteUserPage wired
—	CertificateValidationPage wired to `certificateApi.verify`

Backend Dependencies Summary

Frontend Need	Backend Status	Endpoint	Backend Task
Saved Courses API	❌ Missing	`/api/v1/learning/saved-courses/`	29
Course ordering	❌ Missing	`?ordering=-enrollment_count` on CourseViewSet	30
Org user/course count	❌ Missing	annotations on OrganizationSerializer	31
Course stats	❌ Missing	`/api/v1/catalogue/courses/stats/`	32
Assessment stats	❌ Missing	`/api/v1/superadmin/assessments/stats/`	33
Certificate stats (admin)	❌ Missing	`/api/v1/learning/certificates/stats/`	34
Instructor stats	✅ Exists (partial)	`usersApi.getInstructorStats()`	35
Invoice stats	❌ Missing	`/api/v1/payments/invoices/stats/`	36
Revenue breakdown	❌ Missing	`/api/v1/payments/revenue/stats/`	37
Mobile money charge	❌ Missing	`/api/v1/payments/flutterwave/charge-mobile-money/`	60

Frontend Need	Backend Status	Endpoint	Backend Task
Promo code verify	❌ Missing	`/api/v1/public/promo-codes/verify/`	61
Review helpful/report	❌ Missing	`/api/v1/catalogue/reviews/{id}/helpful/`	62
Transaction export CSV	❌ Missing	`/api/v1/payments/transactions/export-csv/`	63
Manager settings	❌ Missing	`/api/v1/manager/settings/`	43
Manager billing	❌ Missing	`/api/v1/manager/billing/plan/`	44
Security stats	❌ Missing	`/api/v1/superadmin/security/stats/`	22
System health	❌ Missing	`/api/v1/superadmin/system/health/`	38
Public Stats	✅ Working	`/api/v1/public/stats/`	—
Trusted Clients	⚠️ Stubbed	`/api/v1/public/clients/`	—
Storage Quota	✅ Working	`/api/v1/uploads/quota/`	—
Enrollment Trends	✅ Working	`/api/v1/learning/analytics/enrollment-trends/`	—
Learning Stats	✅ Working	`/api/v1/learning/analytics/learning-stats/`	—
Revenue Trends	✅ Working	`/api/v1/payments/analytics/revenue/`	—
Courses by Category	✅ Working	`/api/v1/catalogue/analytics/courses-by-category/`	—
Reports	✅ Working	`/api/v1/learning/reports/`	—
Bulk Enrollment	✅ Working	`/api/v1/learning/enrollments/bulk/`	—
Session Attachments	✅ Working	`/api/v1/catalogue/session-attachments/`	—
Messaging	✅ Working	`/api/v1/messaging/`	—
Badges	✅ Working	`/api/v1/learning/badges/`	—